- ***1*** Takes a single index value from the flattened grid and converts it back into [x,y] coordinates
- ***2*** Finds x coordinate
- ***3*** Finds y coordinate
- ***4*** This is the reward function for the 2D grid.
- ***5*** The reward is based on how different the action is from the mean field action.
- ***6*** Scales the reward to be between [–1,+1] using the tanh function

It is inconvenient to work with [x,y] coordinates to refer to agents in the 2D grid. We generally refer to agents using a single index value based on flattening the 2D grid into a vector, but we need to be able to convert this flat index into [x,y] coordinates, and that is what the `get_coords` function does. The `get_reward_2d` function is our new reward function for the 2D grid. It computes the difference between an action vector and a mean field vector. For example, if the mean field vector is [0.25,0.75] and the action vector is [1,0], the reward should be lower than if the action vector were [0,1]:

```
>>> get_reward_2d(torch.Tensor([1,0]),torch.Tensor([0.25, 0.75]))
tensor(-0.8483)

>>> get_reward_2d(torch.Tensor([0,1]),torch.Tensor([0.25, 0.75]))
tensor(0.8483)
```

Now we need to create a function that will find an agent's nearest neighbors and then compute the mean field vector for these neighbors.

Listing 9.11. Mean field Q-learning: Calculate the mean action vector

```
def mean_action(grid,j):
    x,y = get_coords(grid,j)                          1
    action_mean = torch.zeros(2)                      2
    for i in [-1,0,1]:                                3
        for k in [-1,0,1]:
            if i == k == 0:
                continue
            x_,y_ = x + i, y + k
            x_ = x_ if x_ >= 0 else grid.shape[0] - 1
            y_ = y_ if y_ >= 0 else grid.shape[1] - 1
            x_ = x_ if x_ <  grid.shape[0] else 0
            y_ = y_ if y_ < grid.shape[1] else 0
```